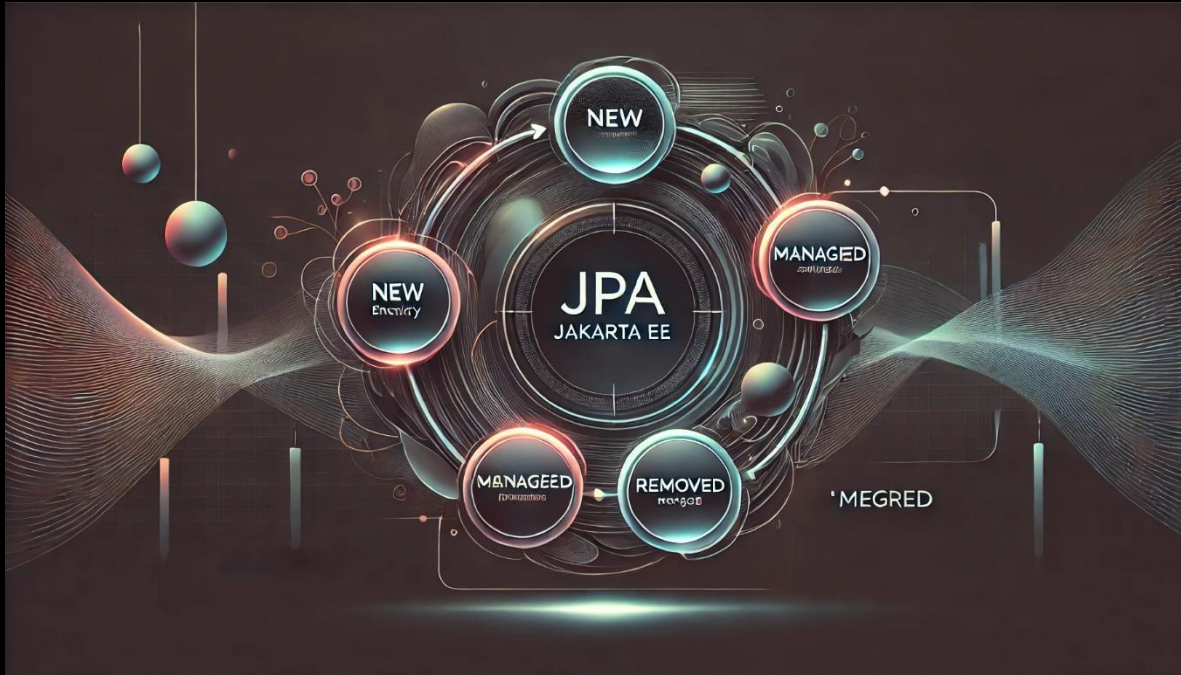


Persistir objetos con JPA y Jakarta EE



Persistencia con JPA y Jakarta EE

➡ 1. Configuración del Proyecto

Trabajaremos con el proyecto **holamundo-jpa**, ya que al ser un proyecto Java puro (no web), nos permite ejecutar métodos `main` sin modificaciones adicionales.

✚ Pasos Iniciales:

1. **Copiar el código necesario:**
 - Pasamos el código de las clases de entidad de Persona y Usuario del paquete de `sga.dominio` del proyecto **sga-jee-web** hacia el proyecto **holamundo-jpa**
2. **Verificar la configuración de `persistence.xml`**
 - Vamos a modificarlo para usar una conexión **local** y no la que se ejecuta en GlassFish.

➡ 2. Modificar `persistence.xml` para conexión local

📌 **Ubicación:** `src/main/resources/META-INF/persistence.xml`

1. Abre `persistence.xml`

2. Cambia el nombre de la **unidad de persistencia** a SgaPU
3. Configura la conexión para que sea local en lugar de JTA.

✦ Código Final de `persistence.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.2"
    xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
        https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd">
    <persistence-unit name="SgaPU" transaction-type="RESOURCE_LOCAL">
        <class>sga.domain.Persona</class>
        <class>sga.domain.Usuario</class>
        <properties>
            <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/test?useSSL=false&useTimezone=true&serverTimezone=UTC" />
            <property name="jakarta.persistence.jdbc.user" value="root" />
            <property name="jakarta.persistence.jdbc.password" value="admin" />
            <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver" />
            <property name="eclipselink.logging.level.sql" value="FINE" />
            <property name="eclipselink.logging.parameters" value="true" />
            <property name="eclipselink.logging.timestamp" value="false"/>
        </properties>
    </persistence-unit>
</persistence>
```

◆ Explicación:

- `transaction-type="RESOURCE_LOCAL"` → Indica que trabajamos con transacciones locales.
- `jakarta.persistence.jdbc.url` → Dirección de la base de datos MySQL local.
- `hibernate.show_sql` y `hibernate.format_sql` → Permiten ver las consultas SQL generadas en la consola.

➡ 3. Crear la Clase `PersistirObjetoJPA`

📍 **Ubicación:** `src/main/java/mx/com/gm/sga/cliente/ciclodevida`

1. Crea una nueva clase en el paquete `mx.com.gm.sga.cliente.ciclodevida`
 - o Nombre: `PersistirObjetoJPA`

2. Código de la clase

```
package sga.cliente.ciclodevida;

import jakarta.persistence.*;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class PersistirObjetoJPA {

    private static final Logger log = LoggerFactory.getLogger(PersistirObjetoJPA.class);

    public static void main(String[] args) {
        System.out.println("comenzando");
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
        try (EntityManager em = emf.createEntityManager()) {
            EntityTransaction tx = em.getTransaction();
            //Inicia la transaccion

            //Paso 1. Crea nuevo objeto
            //Objeto en estado transitorio
            Persona persona1 =
                new Persona("Pedro", "Luna", "pluna@mail.com", "55998877");
            //Paso 2. Inicia transaccion
            tx.begin();
            //Paso 3. Ejecuta SQL
            em.persist( persona1 );
            log.info("Objeto persistido - aun sin commit:" + persona1);
            //Paso 4. commit/rollback
            tx.commit();
            //Objeto en estado detached
            log.info("Objeto persistido - estado detached:" + persona1);
            //Cerramos el entity manager
        }

    }

}
```

4. Explicación del Código

✓ Paso 1 – Crear el `EntityManagerFactory`

```
EntityManagerFactory emf =  
Persistence.createEntityManagerFactory("SgaPU");  
EntityManager em = emf.createEntityManager();
```

◆ Se obtiene la conexión a la base de datos basada en la configuración de `persistence.xml`.

✓ Paso 2 – Iniciar una transacción

```
EntityTransaction tx = em.getTransaction();  
tx.begin();
```

◆ Todas las operaciones con JPA requieren estar dentro de una transacción.

✓ Paso 3 – Crear un Objeto en estado `TRANSIENT`

```
Persona personal = new Persona("Pedro", "Luna", "pluna@mail.com",  
"5544332211");
```

◆ **Estado `TRANSIENT`:** El objeto aún no se ha almacenado en la base de datos.

✓ Paso 4 – Persistir el Objeto en la Base de Datos

```
em.persist(personal);
```

◆ Ahora el objeto pasa a estado `MANAGED`, pero aún no se guarda.

✓ Paso 5 – Hacer `commit` de la Transacción

```
tx.commit();
```

◆ Aquí se genera el **ID** del objeto y se almacena en la base de datos.

✓ Paso 6 – Verificar la Clave Primaria

```
log.debug("Objeto persistido con ID: " + personal.getIdPersona());
```

◆ La clave primaria ahora se asigna automáticamente.

✓ Paso 7 – Cerrar `EntityManager`

```
em.close();
```

♦ **IMPORTANTE:** Siempre cerrar el `EntityManager` para liberar recursos, o usar `try-with-resources` para que se cierre el recurso de manera automática como lo hacemos en el código.

➔ 5. Ejecutar la Prueba

✂ Pasos para ejecutar el código:

1. **Verificar la conexión a MySQL:**
 - Asegúrate de que MySQL está corriendo.
 - En **MySQL Workbench**, verifica que existe la base de datos `sga_db`.
2. **Ejecutar la clase `PersistirObjetoJPA`**
 - En NetBeans, haz clic derecho sobre `PersistirObjetoJPA.java`.
 - Selecciona **Run File**.

```
[main] INFO sga.cliente.ciclodevida.PersistirObjetoJPA - Objeto
persistido - aun sin commit:Persona{idPersona=null, nombre=Pedro,
apellido=Luna, email=pluna@mail.com, telefono=13135566}
```

```
[EL Fine]: sql: INSERT INTO PERSONA (APELLIDO, EMAIL, NOMBRE, TELEFONO)
VALUES (?, ?, ?, ?)
```

```
bind => [Luna, pluna@mail.com, Pedro, 13135566]
```

```
[main] INFO sga.cliente.ciclodevida.PersistirObjetoJPA - Objeto
persistido - estado detached:Persona{idPersona=4, nombre=Pedro,
apellido=Luna, email=pluna@mail.com, telefono=13135566}
```

3. **Ver los resultados en la base de datos**
 - En **MySQL Workbench**, ejecuta la consulta:

```
SELECT * FROM persona;
```

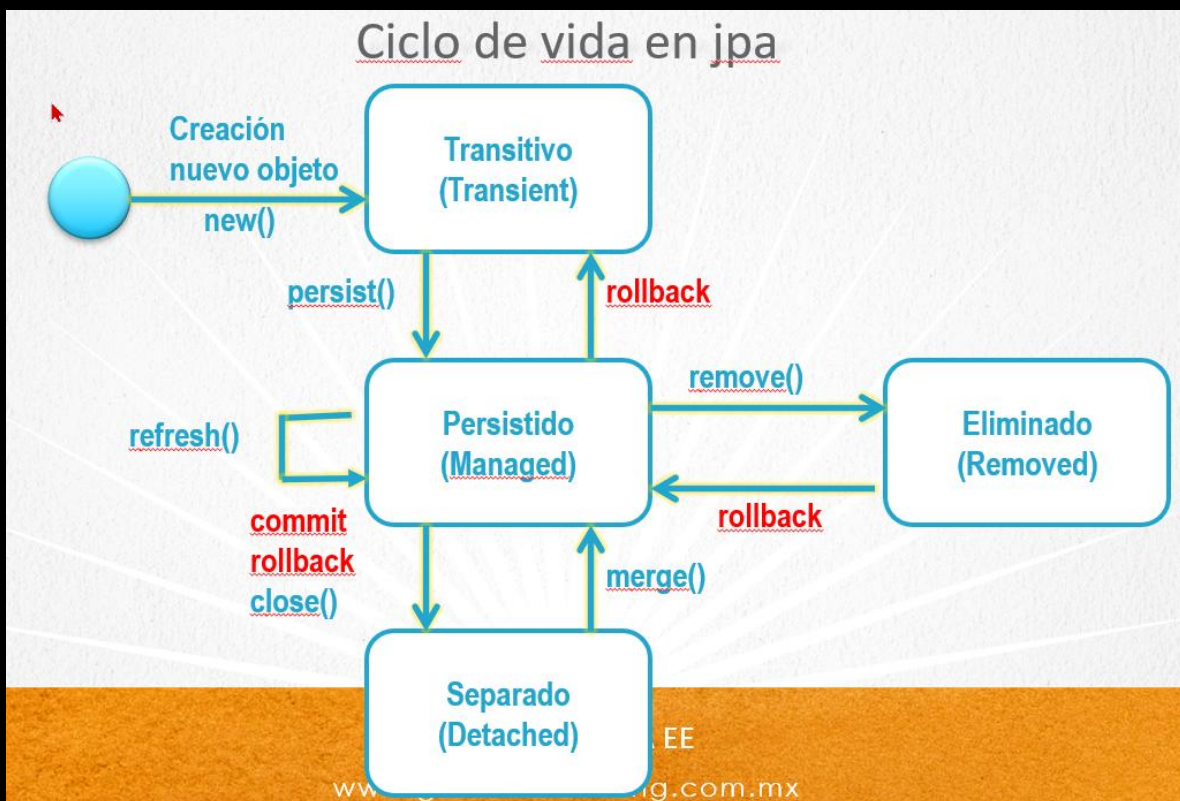
- Verás que el nuevo registro ha sido agregado con su `idPersona` asignado automáticamente.

	id_persona	nombre	apellido	email	telefono
	1	Rafael	Guzman	rguzman@mail.com	55667788
	2	Ivonne	Lara	ilara@mail.com	55443322
	3	Maria	Esparza	mesparza@mail.com	55112233
	4	Pedro	Luna	pluna@mail.com	55998877
➤*	NULL	NULL	NULL	NULL	NULL

➔ 6. Explicación del Ciclo de Vida en JPA

✦ Estados de un Objeto en JPA:

1. **TRANSIENT:**
 - Se crea el objeto, pero no está asociado con la base de datos.
2. **MANAGED:**
 - El objeto es **persistido** en la base de datos, pero aún no se ha confirmado la transacción.
3. **DETACHED:**
 - Si el `EntityManager` se cierra, el objeto ya no está administrado.
4. **REMOVED:**
 - Si se usa `em.remove(objeto)`, el objeto será eliminado de la base de datos.



➔ 7. Conclusión

- ✓ **JPA** nos permite manejar la persistencia sin escribir consultas SQL manuales.
- ✓ **El ciclo de vida** de los objetos en JPA es fundamental para entender cómo interactúan con la base de datos.
- ✓ **Integración con MySQL** → JPA facilita la gestión de datos sin usar código JDBC directamente.

🚀 ¡Ejecuta la prueba y revisa cómo el objeto se almacena en la base de datos automáticamente!

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)